In this section you'll learn how to use `HttpClient` to call HTTP APIs. I'll focus on two common pitfalls in using `HttpClient`—socket exhaustion and DNS rotation problems—and show why they occur. In section 33.2 you'll see how to avoid these problems by using `IHttpClientFactory`.

It's common for an application to interact with other services to fulfill its duty. Take a typical e-commerce store, for example. In even the most basic version of the application, you will likely need to send emails and take payments using credit cards or other services. You could try to build that functionality yourself, but it probably wouldn't be worth the effort.

Instead, it makes far more sense to delegate those responsibilities to third-party services that specialize in that functionality. Whichever service you use, they will almost certainly expose an HTTP API for interacting with the service. For many services, that will be the *only* way.

RESTful HTTP vs. gRPC vs. GraphQL

There are many ways to interact with third-party services, but HTTP RESTful services are still the king, decades after HTTP was first proposed. Every platform and programming language you can think of includes support for making HTTP requests and handling responses. That ubiquity makes it the go-to option for most services.

Despite their ubiquity, RESTful services are not perfect. They are relatively verbose, which means that more data ends up being sent and received than with some other protocols. It can also be difficult to evolve RESTful APIs after you have deployed them. These limitations have spurred interest in two alternative protocols in particular: gRPC and GraphQL.

gRPC is intended to be an efficient mechanism for server-to-server communication. It builds on top of HTTP/2 but typically provides much higher performance than traditional RESTful APIs. gRPC support was added in .NET Core 3.0 and is receiving many performance and feature updates. For a comprehensive view of .NET support, see the documentation at https://learn.microsoft.com/aspnet/core/grpc.